



## **27. ARQUITECTURAS .NET. ARQUITECTURAS J2EE.**

## **TEMA 27. ARQUITECTURAS .NET. ARQUITECTURAS J2EE.**

### **27.1. INTRODUCCIÓN Y CONCEPTOS**

### **27.2 ARQUITECTURA WEB EN .NET**

### **27.3 ARQUITECTURA WEB EN J2EE**

### **27.4. ESQUEMA**

### **27.5. REFERENCIAS**

### **27.1. INTRODUCCIÓN Y CONCEPTOS**

El desarrollo de aplicaciones, servicios web y otros componentes software tiene hoy en día dos vertientes principales, la plataforma .NET y la plataforma J2EE, o JEE nombre con el que es conocida actualmente al cambiar de versión. La competencia entre estas dos tecnologías es muy fuerte, pues proporcionan soluciones similares fuertemente soportadas por las compañías de uno y del otro bando.

**.NET** es la plataforma de desarrollo propuesta por la empresa Microsoft para el mundo de los servidores de aplicaciones que funciona como herramienta de diseño y programación, además de proporcionar un amplio conjunto de utilidades extendidas de apoyo al desarrollo en este tipo de entornos. Por su parte **JEE**, (en inglés *Java Platform, Enterprise Edition*) o Java EE, es una evolución de la plataforma Java para desarrollar y soportar componentes software según un conjunto de especificaciones de modo que puedan operar en un servidor de aplicaciones, incluyendo también herramientas de diseño y programación.

A pesar de que ambas plataformas persiguen el mismo objetivo, tienen una serie de particularidades o diferencias que se ven acentuadas por las guerras comerciales entre las empresas que las soportan. Mientras JEE

tiene soporte multiplataforma, .NET funciona sólo bajo la familia de Sistemas Operativos Windows. Mientras JEE se basa exclusivamente en el lenguaje Java, en .NET se permiten muchos lenguajes de alto nivel, aunque en la práctica los principales sean C# y VB .NET. JEE lleva más años de experiencia en el mercado, mientras que .NET es más reciente. Asimismo JEE presenta mayor soporte en cuanto a soluciones y posibilidades de software libre, que son muy escasas y de poca calidad en .NET. Con JEE se puede instalar una infraestructura de alto rendimiento de manera completamente gratuita.

## **27.2 ARQUITECTURA WEB EN .NET**

**.NET** es, según la empresa Microsoft, una plataforma para el desarrollo de servidores, clientes y servicios. Representa un conjunto de tecnologías que tienen como núcleo principal el .NET Framework, un marco de desarrollo y componente software que puede instalarse en Sistemas Operativos de la familia Windows (Windows 2003, Vista, Windows 7, ...). Existe una versión adaptada para móviles disponible en Windows Mobile. La norma ISO/IEC 23271 recoge un conjunto funcional mínimo que deben cumplir los productos software desarrollados para que puedan funcionar dentro del marco de trabajo. Ésta y más normas se recogen en los estándares:

- a) **Estándar ECMA-334.** Especificación del lenguaje C#. (2006).
- b) **Estándar ECMA-335.** Especificación del lenguaje de infraestructura común (CLI). (2010).

Por el contrario, otros componentes como ASP .NET, *Windows Forms* o ADO .NET no se encuentran estandarizados. Paralelamente, una vez publicados los documentos de especificación de la arquitectura .NET apareció el **Proyecto Mono** con el objetivo de implementar el marco de trabajo .NET Framework empleando código abierto, para a partir de ahí desarrollar aplicaciones para sistemas UNIX/Linux.

### **27.2.1 .NET Framework**

Marco de trabajo que proporciona el conjunto de herramientas y servicios para el desarrollo de componentes software, aplicaciones, servidores y servicios web. Puede dividirse en tres bloques principales:

- 1) El **Entorno de Ejecución Común** (en inglés *Common Language Runtime* o CLR). Se encarga de la gestión de código en ejecución, control de memoria, seguridad y otras funciones relacionadas con el Sistema Operativo.
- 2) La Biblioteca **de Clases Base** (en inglés *.NET Framework Base Classes*). Realizan la función de API de servicios a disposición de los desarrolladores para tareas como gestión de ficheros, mensajería, procesos en varios hilos, acceso a datos, encriptación, etc.
- 3) **Control de acceso a datos**, que permite realizar las operaciones de acceso a datos a través de clases y objetos del *framework* incluidos en el componente ADO.NET.
- 4) El **Motor de Generación de la Interfaz de Usuario**, que permite crear interfaces para aplicaciones de escritorio o web empleando componentes específicos como ASP.NET para web, *Web forms* para aplicaciones de escritorio o *Web services* para servicios web.

### **27.2.2 Entorno de Ejecución Común (CLR)**

El Entorno de ejecución común o CLR es el encargado de gestionar el código en tiempo de ejecución. De manera análoga a la Máquina virtual de Java este entorno permite ejecutar aplicaciones y servicios web o de escritorio en cualquier cliente o servidor que disponga de este software. A diferencia de la Máquina virtual de Java el soporte de .NET es multilenguaje permitiendo C++, C#, ASP .NET, Visual Basic, Delphi, y muchos otros. Además permite integrar y heredar componentes entre diferentes lenguajes, con mayor o menor fortuna a la hora de sacar beneficio de lenguajes antiguos.

El entorno de desarrollo compila el código fuente en cualquiera de los lenguajes soportados a un código intermedio denominado **CIL** (en inglés *Common Intermediate Language*) de manera análoga al BYTECODE de Java. A este lenguaje intermedia se llega empleando la especificación CLS (en inglés *Common Language Specification*) donde se especifican unas reglas necesarias para crear el código intermedio CIL compatible con el CLR. Asimismo, el CLR dispone de compiladores como JIT (en inglés *Just InTime*) o AOT (en inglés *Ahead Of Time*) adaptados a cada lenguaje.

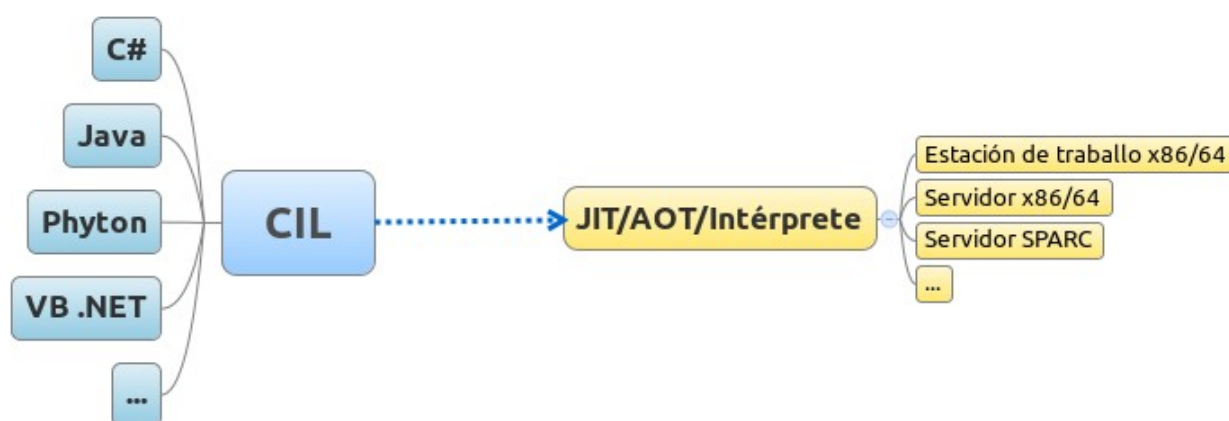
**JIT** genera el código máquina real en cada máquina a partir de ese código intermedio consiguiendo independencia del hardware. Esta compilación se hace en tiempo de ejecución a medida que la aplicación o servicio invoca métodos o funciones. Para agilizar el procesamiento este código máquina obtenido en tiempo de ejecución se guarda en la memoria caché actualizándose tan sólo cuando se produce algún cambio en el código fuente, momento en el que se repite el proceso. Por el contrario, **AOT** compila el código antes de ejecutarse con lo cual logra un mayor rendimiento en ejecución pero menos independencia de la plataforma. En lo tocante a JIT normalmente se distingue entre:

- 1) **Jitter estándar.** Compila el código CIL a nativo bajo demanda.
- 2) **Jitter económico.** No optimiza, traduce cada instrucción así precisa menos tiempo y memoria de compilación.
- 3) **Prejitter.** Realiza una compilación estática de un componente software completo.

Las principales **ventajas** de este modelo de compilación son:

- ✓ La **reutilización** de componentes escritos en diferentes lenguajes en una misma aplicación o servicio web.
- ✓ **Modularidad**, gracias a la implementación del patrón Interfaz para cada componente o librería ya que será accesible desde cualquier lenguaje a través de su API (ASP, C#, Java, Phyton, etc.)

- ✓ **Integración multilenguaje**, ya que cada lenguaje con un compilador a CIL puede integrarse en la plataforma, con lo cual cada componente en ese lenguaje puede integrarse una aplicación o servicio web .NET.
- ✓ **Seguridad**. Por el aislamiento del código de usuario respecto de los accesos a datos y otras partes críticas del Sistema Operativo.



**Figura 1: Estructura multilenguaje del CLR**

**Traducción Texto Figura 1:** Estación de trabajo

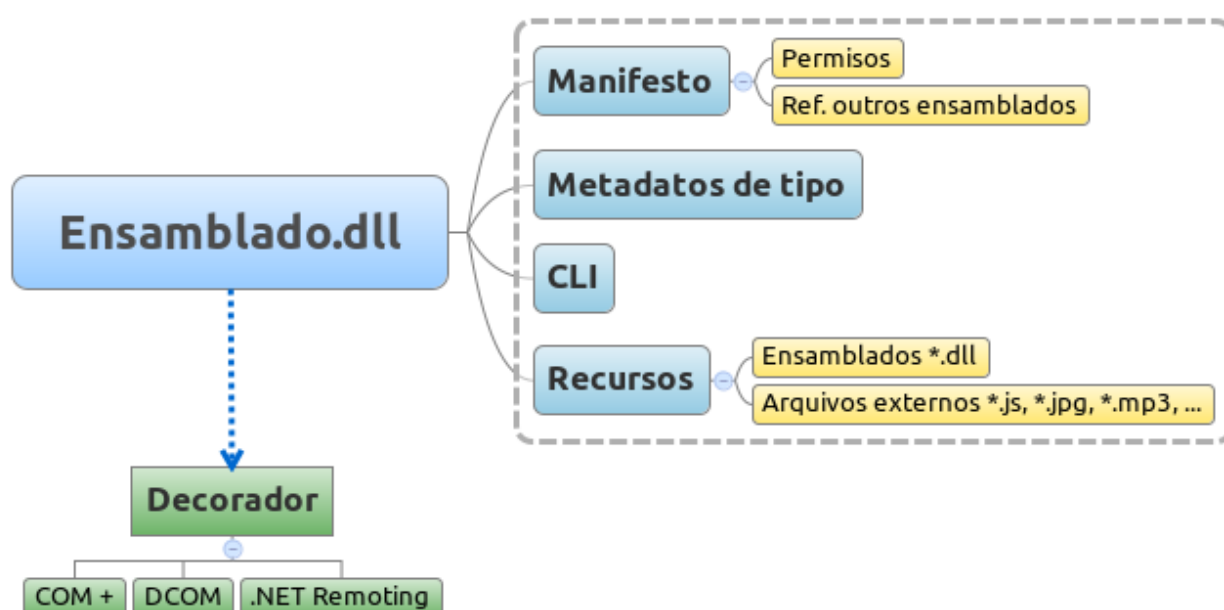
El CLR cumple además la función de proporcionar una amplia gama de servicios a las aplicaciones. A través de la API de cada servicio los componentes web pueden tener acceso a funcionalidades comunes, como:

- a) **Seguridad acceso al código o CAS** (en inglés *Code Access Security*). Controla qué tipo de operaciones puede realizar un código según se identifique en la firma del ensamblado o en el origen del código. Asimismo reconoce directivas de administración del sistema para componentes o a nivel de *host*. Cuando un componente trate de acceder a recursos protegidos del sistema se lanzará el CAS para comprobar los permisos, pero a este nivel no se pueden establecer comprobaciones dinámicas, por ejemplo contra Bases de datos.
- b) **Atributos de protección del host o HPA** (en inglés *Host Protection Attributes*). Mantiene una lista de atributos protegidos, denegando el

acceso o modificación de los mismos. Algunos de estos atributos serían *SharedState*, para estados compartidos, *Synchronization*, para permitir la capacidad de sincronizar procesos en el *host* o *ExternalProcessment* que indica si los procesos en el *host* se pueden controlar externamente a través de la API.

- c) **Dominios de aplicación.** Definen dominios aislados de código para restringir el acceso de los componentes software, creando una zona reservada para un subproceso. Por norma general el CLR crea para cada aplicación un dominio en tiempo de ejecución pero puede precisar dominios específicos para componentes DLL o externos.
- d) **Comprobación de la seguridad de tipos.** Reserva espacios de memoria para cada objeto según lo especificado para su tipo. Cuando el espacio es aleatorio o queda algún hueco fuera del espacio del objeto, quiere decir que ese objeto no tiene seguridad de tipos. Con la compilación JIT se realiza una comprobación en tiempo de ejecución para verificar si cada objeto tiene seguridad de tipos. Del mismo modo impide variables sin valores iniciales o *cast* no seguros.
- e) **Cargador de clases.** Permite cargar en memoria clases y tipos de datos a partir de la interpretación de los metadatos. Existe además la posibilidad de crear cargadores personalizados, aunque sólo para Java, ya que por motivos de rendimiento resulta mucho mejor emplear un ensamblado con otros lenguajes. En la compilación el cargador realiza la función de evitar código innecesario a través de funciones *stubs* que sustituye con el código correcto bajo demanda.
- f) **Recolección de basura** (en inglés *Garbage Collector*). Este servicio se ejecuta de manera continua para buscar y eliminar de memoria los objetos que no sean referenciados o terminen el tiempo de espera de utilización.

- g) **Motor de interacción COM.** Realiza funciones de conversión de datos y mensajes o *marshaling* desde y hacia objetos COM, lo que permite la integración con aplicación *Legacy*.
- h) **Motor de depuración.** Permite realizar un seguimiento de la ejecución del código aunque mezcle diferentes lenguajes.
- i) **API multihilo** (en inglés *multithread*). Proporciona una API y las clases necesarias para gestionar la ejecución de hilos paralelos.
- j) **Gestor de excepciones.** Realiza la gestión estructurada e integración con *Windows Structured Exception Handling* de excepciones aunque el error provenga de diferentes lenguajes en un mismo componente e incluso en el código aún no ejecutado. Este código puede incluir excepciones SHE del tipo C++ o resultados HRESULTS típicos de COM.
- k) **API de la Biblioteca de Clases Base (BCB).** Interfaz con la BCB del marco de trabajo que realiza la integración del código con el motor de ejecución.

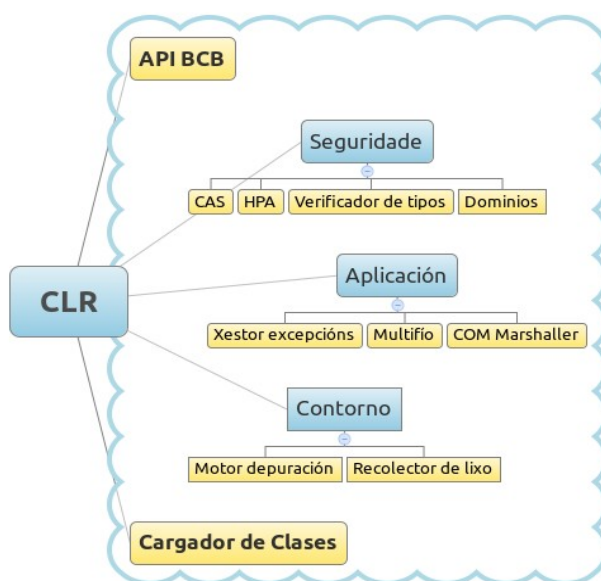


**Figura 2: Ensamblados.**



## **TRADUCCIÓN TEXTO FIGURA 2:** Manifiesto. Otros ensamblados. Archivos externos.

En el .NET Framework cuando se compila un programa o aplicación web se genera un archivo denominado **ensamblado** que contiene el código compilado al lenguaje intermedia CLI y un manifiesto con permisos y referencias a otros ensamblados, componentes software o servicios web. Son paquetes o librerías EXE o DLL destinadas al control de versiones, seguridad y comprobaciones de implementación al por menor. Los ensamblados llevan una indicación que define el entorno de ejecución en el que se debe lanzar: COM+, DCOM, .NET Remoting,...



**Figura 3: Servicios del CLR**

**TRADUCCIÓN TEXTO FIGURA 3:** Seguridad. Gestor de excepciones.  
Multihilo. Entorno. Recogedor de basura.

### **27.2.3 Biblioteca de Clases Base (BCB)**

La Biblioteca de Clases Base es una API de alto nivel para permitir acceder a los servicios que ofrece el CLR a través de objetos en una jerarquía denominada **espacio de nombres**. Agrupa las funcionalidades de uso

frecuente permitiendo su redefinición. Se encuentra implementada en CIL por lo que puede integrarse en cualquier otro lenguaje. Es un conjunto de clases, interfaces y tipos valor que son la base sobre la que se crearán las aplicaciones, componentes y controles del .NET Framework. Permite realizar operaciones como: soporte para diferentes idiomas, generación de números aleatorios, manipulación de gráficos e imágenes, operaciones sobre fechas y otros tipos de datos, integración con APIs antiguas, operaciones de compilación de código adaptada a los diferentes lenguajes de .NET, elementos para interfaces de usuario, tratamiento de excepciones, acceso a datos, encriptación, administración de memoria, control de procesos, etc.

| <b>Espacio de nombres</b>  | <b>Utilidad y objetos</b>                                                                                                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>System</b>              | Tipos básicos, tablas, excepciones, fechas, recolector de basura, etc.                                                                                                                            |
| <b>System.Collections</b>  | Manipulación de colecciones como pilas, colas, <i>hash</i> , etc.                                                                                                                                 |
| <b>System.Data</b>         | Arquitectura ADO.NET (Objetos <i>DataSet</i> , <i>DataTable</i> , <i>DataRow</i> , <i>DataView</i> , ...)                                                                                         |
| <b>System.IO</b>           | Manipulación de Y/S archivos y otros orígenes de datos                                                                                                                                            |
| <b>System.Net</b>          | Gestión de comunicaciones de red (TPC/IP, <i>Sockets</i> , ...)                                                                                                                                   |
| <b>System.Security</b>     | Gestión de las políticas de seguridad del CLR                                                                                                                                                     |
| <b>System.XML</b>          | Acceso y manipulación de datos en documentos XML con compatibilidad con el W3C (Transformaciones en <i>System.Xml.Xls</i> y serialización para servicios web en <i>System.XML.Serialization</i> ) |
| <b>System.Web</b>          | Servicios para gestión de caché, seguridad y configuración para Servicios Web, estado de las sesiones e interfaces de usuario                                                                     |
| <b>System.Web.Services</b> | Gestión de los requerimientos de Servicios Web                                                                                                                                                    |
| <b>System.Web.UI</b>       | Controles para interfaces de usuario <i>HTMLControl</i> para mapeo de etiquetas HTML y <i>WebControl</i> para estructurar controles de usuario avanzados como <i>DataGrids</i>                    |

|                                |                                                                                                                                                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>System.Windows.Forms</b>    | Creación de la IU del cliente                                                                                                                                                     |
| <b>System.Drawing</b>          | Acceso a funcionalidades gráficas básicas de la GDI+ (Funcionalidades avanzadas en <i>System.Drawing.Imaging</i> , <i>System.Drawing.Text</i> y <i>System.Drawing.Drawing2D</i> ) |
| <b>System.Reflection</b>       | Acceso a metadatos sobre los ensamblados, módulos, miembros, parámetros y otras entidades del código administrado                                                                 |
| <b>System.JSON</b>             | Proporciona compatibilidad basada en estándares JSON, nota de objetos JavaScript (en inglés <i>JavaScript Object Notation</i> )                                                   |
| <b>System.Threading</b>        | Manipulación de procesos e hilos de ejecución                                                                                                                                     |
| <b>System.Text</b>             | Proporciona clases para manipular la codificación de caracteres UNICODE y UTF-8 conversión de bloques de caracteres en bloques de <i>bytes</i> y viceversa                        |
| <b>System.Transactions</b>     | Contiene clases que permiten crear y administrar transacciones, admitiendo participantes distribuidos, notificaciones de fase e inscripciones duraderas                           |
| <b>System.Resources</b>        | Proporciona clases e interfaces que permiten crear, almacenar y administrar recursos de localización                                                                              |
| <b>System.Runtime.Remoting</b> | Proporciona la interfaz para acceso remoto y marco para la implantación de sistemas de componentes distribuidos                                                                   |
| <b>Microsoft.CSharp</b>        | Clases para realizar la compilación y ejecución de código en C# (Lo mismo para otros lenguajes)                                                                                   |

**Tabla 1: Principales espacios de nombres.**

#### **27.2.4 Control de acceso a datos.**

El control de acceso a datos, documentos XML y servicios de datos en el marco .NET Framework se recoge en la arquitectura ADO .NET, como evolución del *ActiveX Data Objects*. Su orientación principal es el acceso a datos del gestor de base de datos relacional *SQL Server*, orígenes XML y orígenes de datos vía objetos OLE DB y ODBC. Las **conexiones** se realizan identificando los proveedores de datos a través de los objetos (Connection, Command, DataReader y DataAdapter). Una vez establecida la conexión

entra en escena el principal elemento del marco, el *Dataset*, que recoge los **resultados** cargados a partir de un origen. A su vez, puede particularizarse con otros elementos de la base de datos con objetos como: *DataTable*, *DataRow*, *DataColumn* o *Constraint*. Los **objetivos** de diseño principales de este marco son:

- ✓ Soporte a la tecnología ADO previa.
- ✓ Integración con tecnologías basadas en XML.
- ✓ Soporte a modelos de arquitectura multicapa.

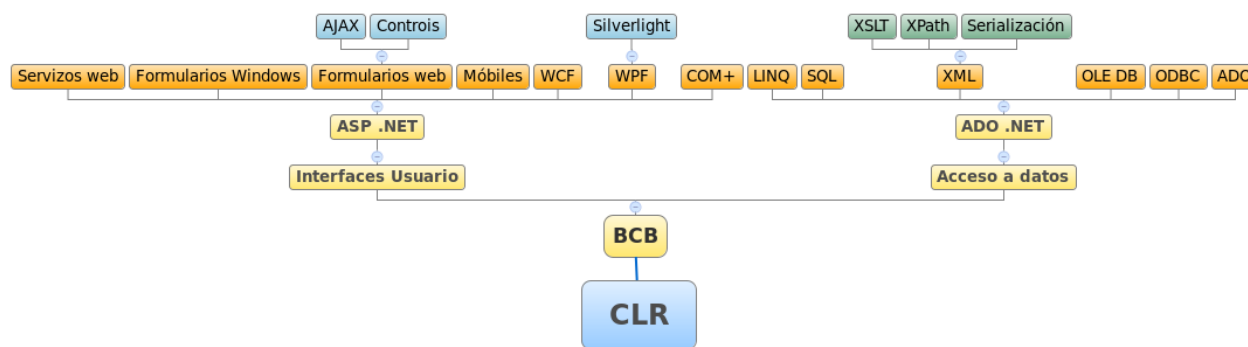
En las últimas versiones se incorpora el **Marco de Entidades** (en inglés *Entity Framework*) que permite realizar Consultas Integradas en los Lenguajes (en inglés *Language Integrated Query*) o **LINQ**.

Este marco permitirá emplear LINQ sobre muchos componentes de acceso a datos nuevos: *LINQ to SQL*, *LINQ to DataSet* y *LINQ to Entities*. Asocia una llave lógica a las entidades, dotando al modelo relacional o conceptual de orientación a objetos. Utilidades del marco de trabajo como **SQLMetal** permiten la generación automática de clases a partir de la base de datos o documentos XML.

### **27.2.5 Motor de Generación de Interfaces de Usuario**

La parte del marco de trabajo encargado de la generación de interfaces de usuario y servicios web sería la arquitectura ASP .NET. El conjunto de clases correspondientes se agrupan en los espacios de nombres: System.Web, System.Web.Services, System.WebUI. Las páginas web desarrolladas con ASP .NET tienen la extensión **ASPX** y son conocidas con **Formularios Web** (en inglés *Web Forms*). Paralelamente a esta tecnología de presentación encontraríamos **Formularios Windows** (en inglés *Windows Forms*) para aplicaciones de escritorio y tecnologías para Móviles, siendo la primera la más empleada para aplicaciones de servidor. En el espacio System.Web.UI

se recogen las dos clases principales de controles los HTML para acceso directo a las etiquetas estáticas de estos lenguajes y los controles web que incorporan código de servidor dinámico. La arquitectura recomendada emplea el modelo **Code-Behind** en el que se crea un archivo separado con el código de servidor, a diferencia de la arquitectura DNA para ASP anterior. Los **Controles de usuario** (en inglés *User Controls*) siguen la misma estructura que los Formularios Web, pero derivan del espacio `System.Web.UI.UserControl`, y se guardan en archivos **ASCX**, que deberían seguir también el modelo Code-Behind. El marco incorpora también elementos para control del estado y la sesión así como otros para seguridad, autenticación de usuarios, uso de papeles, uso del servicio Indigo o WCF (en inglés *Windows Communication Foundation*). Asimismo, permite la integración con AJAX, a través de la incorporación de un **Toolkit** en la aplicación, o WPF (en inglés *Windows Presentation Foundation*) basado en XALM y marco para Silverlight.



**Figura 4: Arquitectura general**

### 27.2.6 Niveles lógicos.

En los modelos y soluciones que propone esta estructura se establecen una serie de servicios genéricos presentes en la mayoría de las aplicaciones corporativas actuales. Esta división permite definir un diseño y una arquitectura específicos para cada nivel, facilitando el desarrollo y soporte de la aplicación.

1. **Servicios de usuarios.** Se encuentran en la primera línea de interacción con los usuarios y proporcionan la interfaz de acceso al sistema que deriva en llamadas a los componentes del nivel de Servicios corporativos. En ocasiones se consideran dentro de este nivel procesos fuera de las interfaces de usuario, como procedimientos de control o automatizados que no requieren la presencia de un usuario.
2. **Servicios corporativos.** Encapsulan la lógica corporativa proporcionando una API de las funcionalidades básicas del sistema. Esto permite abstraer los servicios de usuario de la lógica corporativa y mantener diferentes servicios de usuario a partir de las mismas funcionalidades. Cada funcionalidad puede precisar disponer de varios servicios corporativos.
3. **Servicios de datos.** Sería la parte más aislada del usuario, proporcionando el acceso a datos y a otros sistemas o servidores. Establecen diferentes API genéricas de las que pueden hacer uso los Servicios corporativos. Contienen una amplia gama de orígenes de datos y sistemas de servidor, encapsulando reglas de acceso y formatos de datos.

#### **27.2.6 Soluciones de integración.**

Con la tecnología .NET existen tres principales planteamientos de soluciones arquitectónicas:

- 1) **SOA** (en inglés *Service Oriented Architecture*). Entiende la comunicación entre aplicaciones y componentes como servicios, no necesariamente servicios web, demandados por clientes o suscriptores y proporcionados y publicados por proveedores.
- 2) **MOA** (en inglés *Message Oriented Architecture*). La comunicación se

realiza por paso de mensajes forzando un modelo SOA distribuido.

- 3) **EAI** (en inglés *Enterprise Integration Application*). Especifica una serie de requerimientos de integración y comunicación en sistemas regulados por los patrones de integración Mediación y Federación, donde un sistema EAI hace funciones de *Hub* o *bus* de comunicaciones.

## **27.3 ARQUITECTURA WEB EN J2EE**

JEE representa un conjunto de especificaciones para plataformas de desarrollo basadas en lenguaje Java para servidores de aplicaciones en arquitecturas de múltiples capas. El **JCP** (en inglés *Java Community Process*) es el organismo encargado de validar los requisitos de conformidad para cada plataforma y aceptarla. Las plataformas JEE constan de los siguientes componentes:

- 1) Un conjunto de especificaciones.
- 2) Un test de compatibilidad o CTS (en inglés *Compatibility Test Suite*)
- 3) Una implementación de referencia para cada especificación.
- 4) Un conjunto de guías de desarrollo y buenas prácticas denominadas JEE Blueprints.

Las principales **especificaciones** que incluye JEE dan soporte a Servicios web, RPC basado en XML, Mensajería XML, despliegues, servicios de autorización, conexión remota RMI, JSP, JSF, JSTL, Servlets, Portlets, Applets, JavaBeans, esquemas XML, acceso a datos JDBC, documentación Javadoc, transformaciones XSL, etc.

El soporte multiplataforma del Java tiene su base en la **Máquina Virtual** (VM/JVM o KVM/CVM para móviles), una plataforma lógica capaz de instalarse en equipos con diferente hardware y Sistema operativo e interpretar y ejecutar instrucciones de código **Java bytecode**. La especificación de la VM también se recoge como especificación por la JCP y

del mismo modo están disponibles test de compatibilidad. La forma más habitual para la VM es mediante un compilador JIT pero también permite interpretación. Del mismo modo se permite ejecución segura mediante el modelo de las Java Applets. Programas de cliente que se ejecutan en una VM dentro del navegador después de descargar vía HTTP código del servidor, que se ejecuta en una *Sandbox* muy restringida.

Los componentes software web y de negocio dentro de esta tecnología se despliegan a través de **Contenedores** (en inglés *containers*). Los contenedores son implementaciones de arquitecturas JEE que proporcionan los servicios del servidor de aplicaciones a los componentes, incluyendo seguridad, acceso a datos, manejo de transacciones, acceso a recursos, control de estados, gestión del ciclo de vida y comunicaciones entre otros. Antes de ejecutarse un componente software debe configurarse como un servicio JEE y desplegarse dentro de un contenedor. Los principales serían los contenedores web para Servlets y JSP, los contenedores EJB para componentes de la lógica de negocio, y contenedores de aplicaciones cliente y contenedores de Applets para los programas de cliente y código de cliente para el navegador respectivamente.

Los principales **servicios** que proporciona JEE junto con sus respectivas API serían:

- 1) **HTTP y HTTPS**. Para control de las comunicaciones web y SSL a través de estos protocolos. Las API de servidor vienen dadas por los paquetes de clases Servlets y JSP y la de clientes en el paquete Java.Net.
- 2) **JDBC** (en inglés *Java Data Base Connection*). API de acceso a datos en sistemas gestores de bases de datos relacionales vía SQL. Por un lado aporta la interfaz para ser empleada por los componentes software y por otra la interfaz para que los proveedores puedan desarrollar los controladores específicos. Las versiones más recientes



- son las JDBC 3.0 y 4.0, que incluyen los paquetes *java.sql* y *javax.sql*.
- 3) **JSTL** (en inglés *Java Server Pages Standard Tag Library*). Proporciona las funcionalidades para etiquetas en las páginas JSP.
  - 4) **RMI-IIOP** (en inglés *Remote Method Invocation-Internet Inter-ORB Protocol*). Proporciona la API para permitir comunicaciones en aplicación distribuidas a través de JAVA RMI, por ejemplo para acceder a componentes EJB. Los protocolos más habituales son JRMP, de RMI e IIOP, de CORBA.
  - 5) **IDL** (en inglés *Java Interfaz Definition Language*). Permite la comunicación de clientes con servicios CORBA a través del protocolo IIOP, servicios SOAP o RPC.
  - 6) **JNDI** (en inglés *Java Naming and Directory Interfaz*). Proporciona el servicio de nombres y directorios, indicando el contexto de cada objeto y las relaciones entre ellos. Se divide en dos interfaces, la API de programación y una SPI que permite conectar con proveedores de servicios de nombres y directorios siendo los principales LDAP, CORBA y RMI.
  - 7) **JAXP** (en inglés *Java API for XML Processing*). Soporta el procesamiento de documentos XML que cumplan con los esquemas del W3C a través de DOM, SAX y XSLT.
  - 8) **JMS** (en inglés *Java Message Service*). Proporciona la API de envío de mensajes para comunicarse con un MOM (en inglés *Message-Oriented Middleware*), una abstracción independiente del proveedor para comunicaciones entre sistemas.
  - 9) **JavaMail**. Proporciona la interfaz para controlar el envío y recepción de correos electrónicos. Puede soportar el formato MIME gracias a su integración con marco de trabajo JAF.
  - 10) **JAF** (en inglés *Java Beans Activation Framework*). API que proporciona el marco de trabajo para activación que soporta las peticiones de otros paquetes.
  - 11) **JTA** (en inglés *Java Transaction API*). Orientada hacia el manejo de

transacciones y a permitir la comunicación entre contenedor y componentes del servidor de aplicaciones como los monitores transaccionales y los administradores de recursos.

- 12) **JAX-RPC** (en inglés *Java API for XML-based RPC*). Proporciona soporte para comunicaciones remotas de tipo RPC entre clientes y servicios web con los estándares HTTP y SOAP. Soporta otros estándares como WSDL, así como SSL y TTL para autenticación. El SAAJ (en inglés *SOAP with attachments API for Java*) añade la posibilidad de archivos o notas aportados con los mensajes.

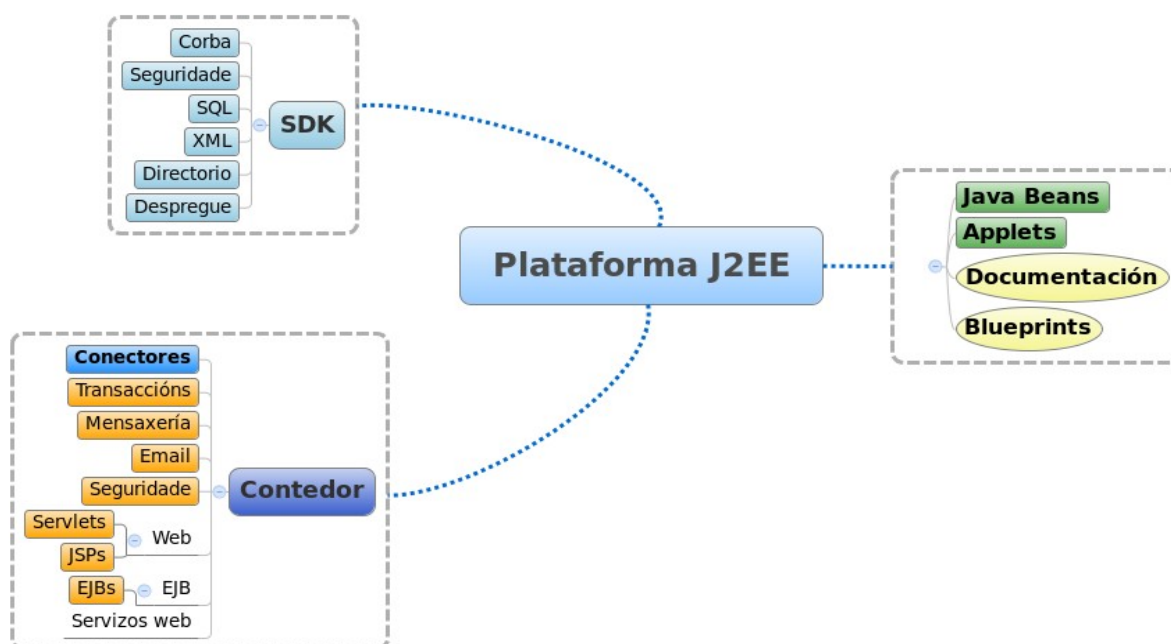
Cada componente se denomina **Módulo** JEE de modo que una aplicación estará formada por un conjunto de módulos siendo cada uno un componente para un contenedor. Existen tres tipos de módulos:

- 1) **Archivos JAR** (en inglés *Java Archive*). Agrupación de archivos Java y recursos según el formato ZIP. Empaquetan componentes EJB según la estructura de directorios del código, añadiendo una carpeta especial, META-INF, con metadatos.
- 2) **Archivos WAR** (en inglés *Web Application Archive*). Agrupan en un único archivo una aplicación web, incluyendo Servlets, archivos JSP, contenido estático y otros recursos web.
- 3) **Archivos EAR** (en inglés *Enterprise Application Archive*). Agrupa en un único archivo varios módulos de una aplicación como archivos WAR o componentes EJB y otras librerías en archivos JAR empaquetados con sus respectivos recursos. Asimismo se incluye el descriptor de despliegue de la aplicación en la carpeta META-INF.
- 4) **Archivos RAR** (en inglés *Resource Adapter Archive*). Contiene un adaptador de recursos de manera análoga a un controlador JDBC y similar a los EAR, pudiendo ir contenido en un archivo de este tipo. El formato viene definido en la especificación JCA (en inglés *Java EE Connector Architecture*).

De manera general conviene considerar a la plataforma como JEE, si bien, existen diferentes **ediciones**, siendo las principales:

- 1) **J2ME**. (en inglés *Java 2 Platform Micro Edition*). Para desarrollo de aplicaciones para dispositivos móviles, electrodomésticos y equipos PDA. Se desarrolló mediante el JPC bajo a especificación JSR 68.
- 2) **J2SE**. (en inglés *Java 2 Platform Standard Edition*). Para desarrollo de aplicaciones de uso general en estaciones de trabajo. Se desarrolló mediante el JPC bajo diferentes especificaciones según las versiones existentes: 1.4, 5.0 y 6.
- 3) **J2EE**. (en inglés *Java 2 Platform Enterprise Edition*). Para desarrollo de aplicaciones destinadas a servidores de aplicaciones para dar soporte a sistemas distribuidos en N capas. Estandarizada por el JPC a partir de la versión 1.4 suele denominarse JEE.

Para cada edición puede distinguirse entre la SDK (en inglés *Software Development Kit*), con el software y recursos destinados al desarrollo de aplicaciones y el JRE (en inglés *Java Runtime Environment*) con el entorno y librerías principales para permitir la ejecución de las aplicaciones.



**Figura 5: Plataforma J2EE.**

**TRADUCCIÓN TEXTO FIGURA 5:** Seguridad. Despliegue. Transacciones. Mensajería. Seguridad. Contenedor. Servicios Web.

### **27.3.1 Modelo de desarrollo**

El modelo de desarrollo más habitual en la arquitectura JEE es un modelo separado en múltiples capas siendo el habitual un mínimo de tres, pero pudiendo llegar a 5 ó 7 según la complejidad del sistema. El objetivo es minimizar el solapamiento entre ellas para que los cambios y modificaciones se limiten al mínimo necesario con el ideal de que se pueda cambiar una de las múltiples capas sin tener que modificar el resto. Se mejora la sostenibilidad, crecimiento del sistema y la reutilización de componentes en el mismo y entre sistemas. Asimismo se permite una mayor heterogeneidad de clientes o elementos de cliente y presentación al modificar sólo la capa más próxima al usuario. El diseño del modelo es análogo a soluciones en .NET pero la implantación diferente. Las 5 capas más habituales se describirán a continuación.

#### **27.3.1.1 Capa de cliente.**

Agrupar los elementos de la interfaz de usuario más próximos al cliente. Ejemplos de estos elementos serían el código (X)HTML/XML y Javascript, los Applets, archivos de recursos y tecnologías RIA. Los tipos de aplicaciones cliente más habituales serían los navegadores web, las aplicaciones de escritorio y actualmente cobran fuerza las aplicaciones para dispositivos móviles. Un aspecto importante en este modelo es garantizar que los EJB de la lógica de negocio sean accesibles tan sólo desde interfaces remotas a través del patrón *SessionFacade*. La variedad de interfaces actual hace que aparezcan *frameworks* de generación dinámica de los mismos basados en el lenguaje **XUL** (en inglés *XML User-Interface Language*), basado en XML. Permite incrustar XHTML y otros lenguajes como MathML o SVG además de CSS. Existen varias alternativas de librerías XUL como Luxor, XWT, Thinlets

o SwingML.

#### **27.3.1.2 Capa de presentación.**

Contiene toda la lógica de interacción directa entre el usuario y la aplicación. Se encarga de generar las vistas más acomodadas para mostrar la información a través de formatos y estilos adecuados. Se componen de una serie de Servlets y páginas JSP que se encargan de devolver el código que irá a la capa de cliente después de comunicarse con la capa de lógica de negocio para obtener los resultados. Puede localizarse en una aplicación de escritorio o en un contenedor web. Además de ensamblar las diferentes vistas, controla el flujo de navegación y hace funciones de autenticación, permisos de acceso y autorización de usuarios, etc. El patrón más habitual en esta capa será el MVC (en inglés *Model View Controller*). Entre las tendencias actuales se encuentran implementaciones de este modelo como Swing el JFace para aplicaciones de escritorio, mientras que dentro de los *frameworks* web se encontrarían Struts, JSF, Tapestry, Expresso y muchos otros, siendo Struts una especie de estándar de hecho. Estos *frameworks* además incorporan otros servicios como etiquetas personalizadas JSTL para interfaces de usuario, manejo de XML, acceso a datos, modelo, filtros, etc.

#### **27.3.1.3 Capa de lógica de negocio.**

Contiene los componentes de negocio reutilizables EJB o POJO, que representan el conjunto de entidades, objetos, relaciones, reglas y algoritmos del dominio o negocio en el que opere el sistema. En esta capa la solución POJO es una opción sencilla que puede mezclar elementos de las capas de integración y datos, mientras que los EJB distinguen los objetos de sesión y las entidades, recomendado en los Blueprints de Sun, emplazando cada uno en su correspondiente capa. El patrón básico en esta capa será el *SessionFacade* donde un único Bean de sesión se encarga de recibir las llamadas de cliente-presentación y dirigir las dentro del contenedor de EJB aislando esta capa.

De manera análoga se establece el modelo de los Bean de mensajería, que realizan comunicación asíncrona mediante JMS en un servidor MOM (en inglés *Messaging Oriented Middleware*), que puede ser un servidor externo al servidor de aplicaciones. También funciona con un patrón Fachada centralizando las llamadas remotas.

#### **27.3.1.4 Capa de Integración.**

Agrupar los componentes encargados del acceso a datos, sistemas *legacy*, motores de reglas de *workflow*, acceso a LDAP, etc. Pueden realizar cambios de formato en la información, pero transformaciones más complejas deberían realizarse en la capa de lógica de negocio, restringido esta a la lógica de acceso a datos o DAO (en inglés *Data Access Objects*) y los encapsuladores de datos y entidades VO (en inglés *Value Object*). Los VO pueden implementarse como POJO o EJB de entidad. Como ocurría anteriormente en el caso de los POJO, se gana en facilidad pero se pierden servicios y funcionalidades como los de persistencia. Los EJB suman complejidad, pero mejoran el rendimiento en memoria. En lo relativo al acceso a datos aparecen las siguientes alternativas:

- 1) **JDBC.** Para POJO o Beans de entidad con control de persistencia. Es una solución sencilla, con pocas funcionalidades pero que hace uso de una API de uso extendido.
- 2) **DAO.** Va un paso más allá que el JDBC incorporando interfaces para abstraer el acceso a datos y hacerlo independiente del lenguaje del gestor. Cada interfaz tendrá una implementación diferente para cada gestor de bases de datos.
- 3) **Frameworks de persistencia.** Hacen las funciones de motores de correspondencia de objetos a bases de datos relacionales definiendo entidades y relaciones vía XML. Realizan gran parte de las funciones de acceso a datos automáticamente. Las soluciones de uso más extendido son los *frameworks* Hibernate, iBatis, TopLink, JPA o a través de EJB.

- 4) **JDO** (en inglés *Java Data Objects*). Sistema de persistencia estándar a partir de una especificación JEE, añadiendo además de la correspondencia entre el modelo relacional y los objetos, la posibilidad de permitir definir los objetos sobre la base de datos. Las implementaciones de uso más extendido son OJB, XORM, Kodo JDO o LIDO.

#### **27.3.1.5 Capa de sistemas de información.**

Se encuentra integrada por los sistemas de bases de datos, ficheros, sistemas 4GL, ERP, Data Warehouse, Servicios web y cualesquiera otros sistemas de información de la organización. En esta capa irían los conectadores para diferentes sistemas de información heterogéneos y los propios recursos que integran los sistemas de información. Las soluciones más habituales se enumeran a continuación.

- 1) **JCA** (en inglés *J2EE Connector Architecture*). Define una interfaz de acceso común independiente del sistema, con la misma API para todos. Se basa en el concepto de adaptador de recursos, siendo cada adaptador un controlador específico para un sistema de información. Las operaciones básicas que define la especificación son: gestión de las conexiones de acceso a JCA, seguridad, transacciones, multiproceso, paso de mensajes y portabilidad dentro de los servidores de aplicaciones.
- 2) **JMS** (en inglés *Java Message Service*). El servicio de mensajes Java emplea colas de mensajes para el traspaso de información entre componentes software estableciendo una infraestructura MOM con dos modelos de API, Punto a punto, entre dos únicos clientes o Publicador/subscriptor donde varios clientes según su papel envían o leen mensajes.
- 3) **Servicios web**. Permiten la comunicación entre sistemas heterogéneos a través del acceso a la URL de aplicaciones



empleando protocolos basados en XML como SOAP o SAAJ. Los clientes acceden al servicio a partir de su interfaz definida ante WSDL (en inglés *Web Service Definition Language*) o insertada en algún registro de servicios web.



**Figura 6: Modelo de desarrollo en capas.**

**TRADUCCIÓN TEXTO FIGURA 6:** LÓGICA DE NEGOCIO. Servidor de aplicaciones. Transacciones. Ficheros. Servicios web.

### 27.3.2 Servidores de aplicaciones.

El servidor de aplicaciones será el encargado de soportar la mayoría de las funcionalidades y servicios de la tecnología JEE, siendo el núcleo de esta arquitectura. Cuando un servidor de aplicación implementa la tecnología JEE tiene que proporcionar todos los componentes definidos en la especificación y por tanto cualquier aplicación JEE podrá desplegarse y ejecutarse en el mencionado servidor.

El servidor de aplicaciones dispondrá de diferentes contenedores para Applets y aplicaciones clientes, web y EJB, siendo estos últimos los que se encargarán de operar con la lógica del dominio, gestión de transacciones,



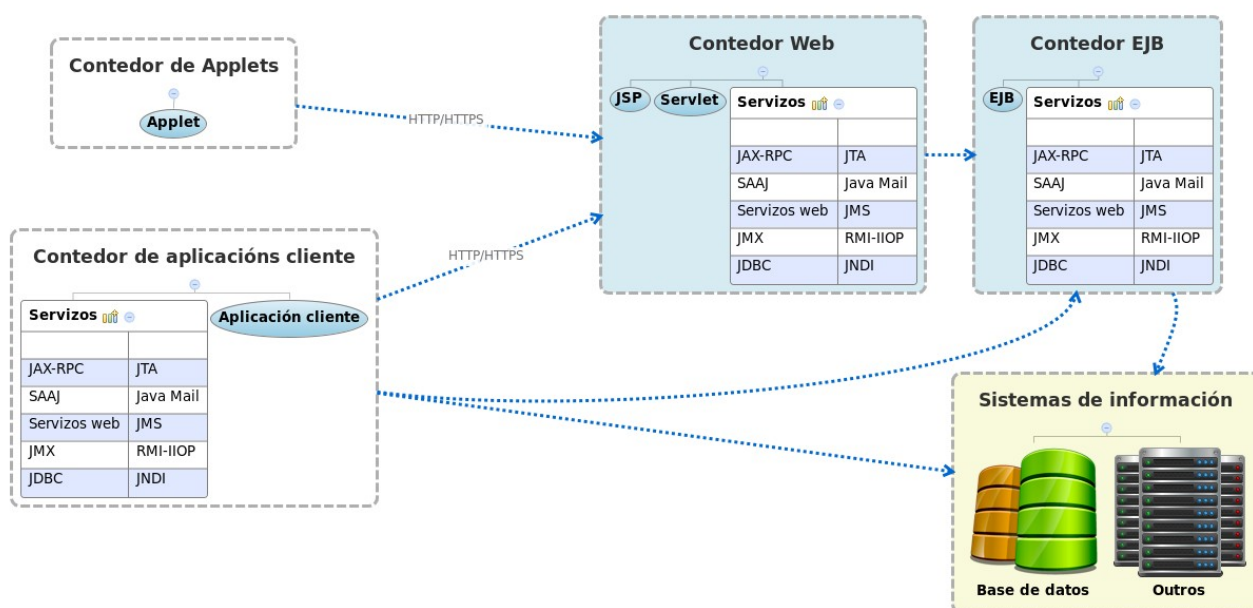
persistencia, control del flujo, etc.

- a) **Tomcat**. Sin presentar todas las funcionalidades de un servidor de aplicaciones, este servidor libre de Apache incorpora el servidor web Apache y soporte para JSP y Servlets con el contenedor Catalina. Presenta diferentes módulos de soporte de aplicación como seguridad SSL, SSO, JMX, AJP, JSF, conector Coyote para peticiones HTTP, soporte para Comet, Recolector de basura reducida así como herramientas web para despliegue y administración.
- b) **JBoss**. Uno de los servidores de aplicaciones libres de uso más extendido compuesto por un Contenedor de Servlets para JSP y Servlets y un Contenedor de Beans. A diferencia de Tomcat implementa todo el conjunto de servicios especificados por JEE. Como contenedor de Servlets emplea una adaptación de Tomcat o el contenedor Jetty. Entre los módulos y funcionalidades que soporta destaca que permite la creación de *cluster*, soporte EJB, JMX, Hibernate, JBoss AOP para dotar a clases Java de persistencia y funcionalidad transaccional, sistema caché, JSF, Portlets, JMS, Servidor de correo, gestión de contenidos foros y portales, entre otras muchas.
- c) **Geronimo**. Otro producto libre de Apache, compatible con JEE que incluye JDBC, RMI, JM, Servicios web, EJB, JSP, Servlets y otras tecnologías. La principal característica de este servidor es que integra un gran número de otras soluciones ya existentes: Tomcat y Embarcadero como contenedores web, OpenEJB como contenedor de Servlets, OpenJPA, Apache Axis, Apache CXF y Scout Apache para servicios web, Derby para el acceso a datos, y WADI para establecer clusters y balanceo de carga, entre otros.
- d) **JonAS**. Otra alternativa libre a JBoss, aunque no soporta por completo JEE. Permite integración con Tomcat o Jetty como contenedores web y tiene contenedor de EJB. Entre módulos y

servicios incorpora: Xplus, Hibernate, TopLink, OpenJPA, JORAM como implementación de JMS, varios protocolos RMI (IIOP, JRMP, IRMI), soporte LDAP, servicios web Axis y otros muchos.

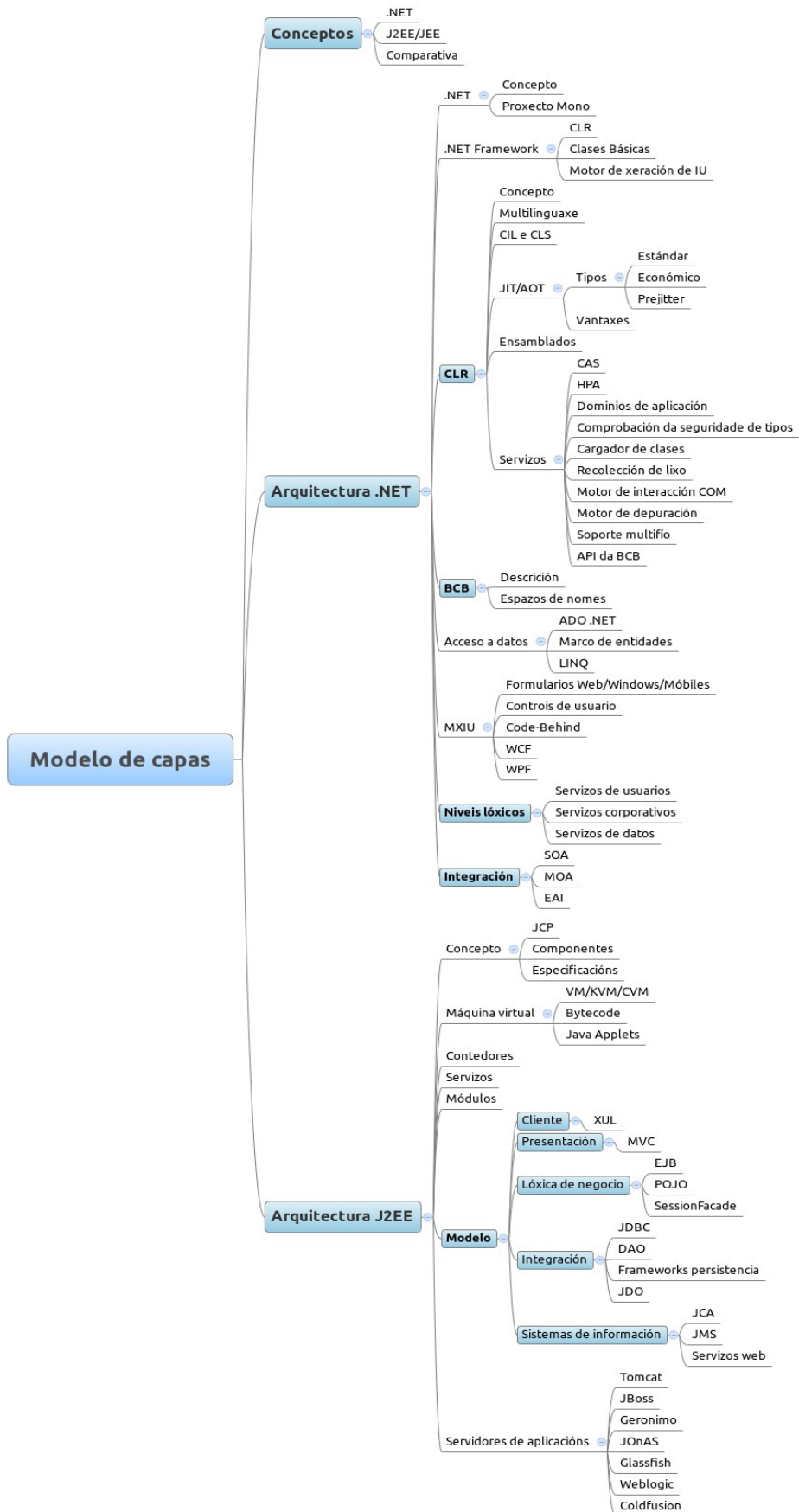
- e) **Glassfish.** Alternativa libre de Sun, ahora Oracle, que tiene como base el *framework* para persistencia Toplink. Incorpora además módulos para soporte EJB, JAX-RS, JSF, RMI, JMS, servicios web, en la línea de los anteriores, y novedades como Apache Félix, una implementación de OSGi (en inglés *Open Services Gateway*) y Grizzly que hace uso de la nueva API de Java de E/S (NIO) para mejorar la escalabilidad.
- f) **WebSphere.** Alternativa comercial de IBM con una versión de libre distribución. La versión libre, más ligera, se basa en el servidor Geronimo diferenciándose de este en que incluye soporte para DB2, Informix, soporte RAC de Oracle y otras bases de datos así como mejores librerías para XML. Otras tecnologías serían: los Servlets SIP (en inglés *Session Initiation Protocol*) que utilizan elementos multimedia en tiempo real, mensajería instantánea y juegos on line; el *framework* Spring; protocolos de seguridad Kerberos y SAML. La diferencia con otros servidores es que posee herramientas de administración más avanzadas, sobre todo para sistemas en cluster y soporte para *mainframes*.
- g) **Weblogic.** Alternativa comercial de Oracle basada en Glassfish, incorporando servicios del Weblogic server sobre la JVM JRockit. En concreto Weblogic Server proporciona los Servicios web Oracle WebLogic Server Services Web, la Application Grid como solución de grid de datos, soporte de conectividad con Tuxedo (WTC), soporte de RAC para Oracle, SAML, una API de integración con .NET a JMS.NET, Spring y el *framework* de diagnóstico WLDF.
- h) **Coldfusion.** Alternativa comercial de Adobe de las más valoradas actualmente, diferenciándose por el soporte a tecnologías RIA

principalmente Flash. Implementa parte de los servicios JEE pero puede integrarse con otros servidores de aplicaciones como WebSphere o Jboss, pudiendo desplegarse cómo aplicación Java. Además lleva incorporado el servidor de aplicaciones Adobe JRun. Destaca por el soporte en tecnologías AJAX, Flex, PDF, RSS, Flash Remoting, integración .NET y herramientas de administración avanzadas.



**Figura7: Arquitectura J2EE**

## 27.4. ESQUEMA



**TRADUCCIÓN TEXTO ESQUEMA:** Proyecto Mono. Motor de generación de IU. Multilenguaje. Ventajas. Comprobación de la seguridad de tipos. Recolección de basura. Soporte multihilo. Descripción. Espacios de nombres. Controles de usuario. NIVELES LÓGICOS. Servicios de usuario. Servicios Corporativos. Servicios de datos. Componentes. Especificaciones. Contenedores. Servicios. Lógica de negocio. Servidores de aplicaciones.

## **27.5. REFERENCIAS**

Varios autores.

Biblioteca MSDN de Microsoft. (2003).

Jef Ferguson y otros.

La biblia de C#. (2003).

Benjamín Aumaille.

J2EE. Desarrollo de aplicaciones Web. (2002).

I. Singh, B. Stearns y otros.

Desingning Enterprise Applications with the J2EE Platform. (2002).

**Autor: Juan Marcos Filgueira Gomis**

**Asesor Técnico Consellería de Educación e O. U.**

**Colegiado del CPEIG**